Study Coach

Documentazione di Progetto

Sviluppatori

Diabi Mohamed Zakaria
Gouaiche Kamal

Documentazione

15 agosto 2026

Indice

1. Descrizione del progetto
2. Architettura del progetto
3. Decisioni prese nel progetto
4. Tecnologie e librerie usate
5. Processo di implementazione
6. Percorsi e configurazione di default
7. Funzionalità implementate
8. Struttura delle cartelle
9. Testing
10. Limitazioni note e possibili sviluppi futuri
11. Divisione dei contributi (Agent)

1. Descrizione del progetto

Study Coach è un'applicazione di supporto allo studio universitario che trasforma il materiale didattico (PDF) caricato dallo studente in un percorso di apprendimento interattivo, basato sul **metodo socratico** invece che sulla semplice correzione domanda/risposta.

L'idea di fondo: invece di dare subito la risposta corretta, il sistema fa ragionare lo studente attraverso domande guida, poi mostra dove ha sbagliato e fornisce la risposta corretta e migliorata — il tutto ancorato esclusivamente al materiale caricato (RAG, *Retrieval-Augmented Generation*), per evitare risposte generiche o allucinate.

Il progetto offre due modalità di interazione:

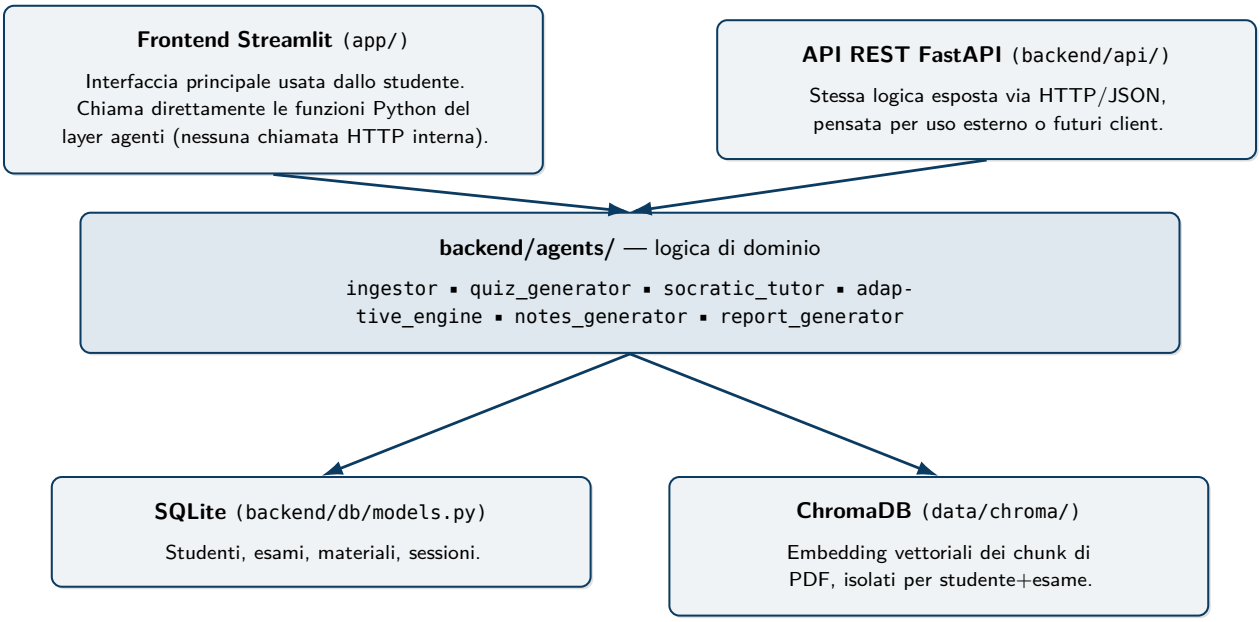
- **Sessione quiz strutturata**: il sistema genera N domande a risposta aperta dal materiale caricato e guida lo studente attraverso ciascuna con il loop socratico.
- **Chatbot dubbi libero**: lo studente scrive un dubbio proprio (non un quiz) e il tutor risponde con una domanda socratica invece di dare subito la soluzione.

Al termine di ogni sessione viene generato un **report** (Markdown + PDF) con punteggio, argomenti deboli/forti e sintesi, e sono disponibili **appunti sintetici** generati dal materiale o dal log della sessione.

---

2. Architettura del progetto

Il progetto è organizzato in tre macro-livelli, con una netta separazione tra logica di dominio e interfacce:



Componenti principali

Livello	Cartella	Ruolo
Frontend	`app/pages/*.py`	Pagine Streamlit (login, upload, exam, session, chat, history). Chiamano direttamente le funzioni di `backend/agents` — nessuna chiamata HTTP interna.
API REST	`backend/api/*.py`	Router FastAPI (`auth`, `exams`, `sessions`) che espone la stessa logica via HTTP/JSON, con stato dei tutor/engine attivi tenuto in memoria di processo.
Logica di dominio	`backend/agents/*.py`	Agenti indipendenti dall'interfaccia: ingestion RAG, generazione quiz, tutor socratico, motore adattivo, generazione appunti/report.
Persistenza	`backend/db/models.py`	Modelli SQLAlchemy (`Student`, `Exam`, `Material`, `Session`) su SQLite (`data/study_coach.db`).
Contratti API	`backend/models/schemas.py`	Modelli Pydantic per richieste/risposte dell'API REST.
Vector store	ChromaDB (`data/chroma/`)	Una collection isolata per ogni coppia `student_id+exam_id` (`student_{id}_exam_{id}`).

Flusso dati principale

1. Lo studente carica un PDF → `ingestor.py` estrae il testo, lo divide in chunk (500 parole, overlap 50) e li indicizza in ChromaDB con embedding locali.
2. `quiz_generator.py` recupera i chunk più rilevanti (RAG) e chiede a un agente LLM (Groq) di generare N domande aperte, calibrate su tre livelli di difficoltà.
3. `socratic_tutor.py` gestisce il loop domanda → prima risposta → domanda guida (senza svelare la risposta) → seconda risposta → valutazione (0-10, con guardrail di coerenza/severità) → eventuale contestazione del voto.
4. `adaptive_engine.py` tiene traccia dei punteggi per argomento e sceglie la prossima domanda dando priorità agli argomenti deboli.
5. A fine sessione, `report_generator.py` produce un report (Markdown + PDF) salvato su disco e collegato alla sessione nel DB.

---

3. Decisioni prese nel progetto

Alcune scelte architetturali rilevanti, con la motivazione:

- **Metodo socratico invece di correzione diretta** — il tutor non dà mai la risposta corretta al primo tentativo: genera sempre una domanda guida. Scelta pedagogica per stimolare il ragionamento invece della memorizzazione passiva.
- **Embedding locali invece di API esterne** — la cronologia dei commit mostra che l'ingestor è partito con API key Gemini per gli embedding (`2d3b8c3`) ed è stato poi migrato a `sentence-transformers` (`all-MiniLM-L6-v2`) eseguito in locale. Motivazione: nessun costo per chunk indicizzato, funzionamento offline, nessun dato del materiale inviato a terzi solo per vettorizzarlo.
- **LLM (Groq/Llama 3.3 70B) usato solo per ragionamento, non per gli embedding** — separazione tra "capire/generare linguaggio" (LLM remoto, veloce e economico su Groq) e "cercare informazioni" (embedding locali), per bilanciare costo, latenza e qualità.
- **Nessun fallback silenzioso sulle chiamate LLM** — le funzioni che interrogano l'agente (`generate_quiz`, `SocraticTutor`, `notes_generator`) ritentano indefinitamente in caso di errore, rate limit o JSON malformato, invece di restituire un risultato posticcio. Scelta esplicita (vedi commenti nel codice) per garantire che ogni domanda/valutazione sia realmente generata dal materiale, mai un placeholder.

- **Guardrail deterministici sopra il giudizio dell'LLM** — `_apply_severity_guardrails` forza il punteggio a 0 se la seconda risposta è una non-risposta ("non so", "boh", ecc.), e `_apply_consistency_guardrails` corregge automaticamente un punteggio insufficiente quando il modello non ha segnalato lacune reali. Motivazione: l'LLM può essere incoerente tra il voto numerico e le motivazioni testuali; queste regole rendono la valutazione riproducibile. *(Presente anche il flag opposto simmetrico: la contestazione del voto usa un secondo agente imparziale — `ContestationAgent` — per rivalutare, non lo stesso agente che ha già giudicato.)*
- **Isolamento dei dati per studente+esame** — sia la collection ChromaDB (`student_{id}_exam_{id}`) sia le query SQL sono sempre filtrate per `student_id`, così materiali ed esami di studenti diversi non si mescolano mai nel recupero RAG.
- **Due interfacce parallele (Streamlit diretto + API REST)** — la UI Streamlit non passa dall'API REST: chiama direttamente le funzioni Python degli agenti. L'API REST esiste come strato aggiuntivo, pensato per un consumo esterno futuro (altri client, integrazioni), e replica la stessa logica tramite router FastAPI.
- **Stato dei tutor/engine attivi in memoria di processo** — `backend/api/sessions.py` tiene `_active_tutors` e `_active_engines` come dizionari in memoria (non persistiti). Implica che un riavvio del server API interrompe le sessioni socratiche in corso: compromesso accettato per semplicità, dato che lo stato definitivo (punteggio, log Q&A) viene comunque scritto su DB alla chiusura di ogni domanda/sessione.
- **Cookie di sessione persistente lato browser** — `backend/session_cookie.py` genera un token opaco per studente e lo scrive come cookie via componente HTML iniettato, per mantenere il login tra un refresh e l'altro di Streamlit (che altrimenti perderebbe lo stato ad ogni riconnessione websocket).
- **Pulizia automatica delle sessioni orfane** — sia `sessions.py` (API) sia `history.py` (UI) eliminano/chiudono automaticamente le sessioni avviate ma mai concluse (`ended_at` nullo, nessun punteggio), per non sporcare lo storico con sessioni abbandonate a metà.

---

4. Tecnologie e librerie usate

Categoria	Scelta	Note
Linguaggio	Python 3.13	`requires-python = ">=3.13"`
Gestione dipendenze	`uv` (`uv.lock`)	con fallback pip
Frontend	**Streamlit** 1.57	UI reattiva single-page-per-stato, CSS custom iniettato per il tema
API REST	**FastAPI** 0.136 + Uvicorn	router modulari (`auth`, `exams`, `sessions`), CORS abilitato per `localhost:8501`
ORM / DB	**SQLAlchemy** 2.0 su **SQLite**	`data/study_coach.db`, migrazione leggera in-app per la colonna `session_token`
Validazione dati	**Pydantic** 2.x	schemi richiesta/risposta dell'API (`backend/models/schemas.py`)
Vector store	**ChromaDB** 1.5	`PersistentClient`, una collection per studente+esame
Embedding	**sentence-transformers** (`all-MiniLM-L6-v2`)	locale, 384 dimensioni, nessuna API key
Estrazione PDF	**pypdf**	estrazione testo pagina per pagina
Generazione PDF	**fpdf2**	report sessione e appunti, con layout Markdown→PDF fatto a mano

Categoria	Scelta	Note
Agenti LLM	**Agno** (framework agenti) + **Groq** (`llama-3.3-70b-versatile`)	agenti specializzati per ruolo: quiz, guida socratica, valutazione, contestazione, dubbi, appunti
Config/segreti	**python-dotenv**	`.env` con `GROQ_API_KEY`, `GEMINI_API_KEY`, `MISTRAL_API_KEY`
Test	**pytest** 9.x + `TestClient` (FastAPI/httpx) + `streamlit.testing.v1.AppTest`	unit, integration ed e2e (vedi [§9](#9-testing))
Codebase intelligence	**graphify**	grafo del codice usato per l'esplorazione/navigazione del repository durante lo sviluppo, in alternativa alla grep testuale

---

5. Processo di implementazione

1. **Setup iniziale** — inizializzazione ambiente, modelli DB, primo agente `ingestor` (con API key Gemini per gli embedding).
2. **Motore di dominio** — aggiunta di `quiz_generator`, `socratic_tutor`, `adaptive_engine` e dei template di prompt versionati (`backend/agents/prompts/*_v1.txt`).
3. **Backend "Agno"** — integrazione del framework di agenti Agno con il modello Groq come backend LLM per tutte le generazioni testuali.
4. **Pagine applicative** — costruzione delle pagine Streamlit per la gestione esami e il loop di sessione.
5. **Hardening del tutor socratico** — pulizia delle sessioni non valutate, guardrail di valutazione più severi/coerenti, gestione della durata sessione.
6. **Refactoring e leggibilità** — riorganizzazione della struttura del codice.
7. **Copertura di test** — introduzione di test unitari e di integrazione per gli agenti backend.
8. **Migrazione embedding** — implementazione e configurazione di `LocalEmbeddingFunction` (embedding locali) al posto della sola dipendenza da API esterne.
9. **Chatbot dubbi libero** — nuova classe `DoubtTutor` e pagina `app/pages/chat.py`: flusso socratico avviato dallo studente invece che dal quiz, con due nuovi prompt dedicati.
10. **Suite di test completa** — 157 test tra unit, integration (`FastAPI TestClient`) ed e2e (`streamlit.testing.v1.AppTest`), con isolamento totale da DB/ChromaDB/reports reali tramite fixture pytest dedicate (`test/conftest.py`) e un `FakeAgent` che sostituisce le chiamate LLM reali.

---

6. Percorsi e configurazione di default

Percorsi su disco (relativi alla root del progetto)

Percorso	Contenuto	Creato da
`data/study_coach.db`	Database SQLite (studenti, esami, materiali, sessioni)	`backend/db/models.py` (`init_db()`)
`data/chroma/`	Vector store ChromaDB persistente	`backend/agents/ingestor.py`

Percorso	Contenuto	Creato da
`data/uploads/`	PDF caricati dagli studenti (salvati con prefisso `s{student_id}_e{exam_id}_`)	`backend/api/exams.py`, `app/pages/exam.py`
`reports/`	Report sessione generati (`{student_id}_{exam_id}_{ses` `md/.pdf`)	`backend/agents/report_generator.py`
`backend/agents/prompts/*.txt`	Template dei prompt per ciascun agente, versionati (`_v1`)	statico, letto a runtime
`graphify-out/`	Grafo del codice generato da graphify (analisi/navigazione repo)	strumento esterno, non parte del runtime applicativo

Variabili d'ambiente (`.env`)

```
GROQ_API_KEY=...     # obbligatoria: LLM per quiz/tutor socratico/appunti
GEMINI_API_KEY=...   # presente per compatibilità storica / uso opzionale
MISTRAL_API_KEY=...  # backend LLM alternativo opzionale
```

Porte e host di default

Servizio	Comando	Porta di default
Frontend Streamlit	`streamlit run app/main.py`	`8501`
API REST FastAPI	`uvicorn backend.api.main:app --reload`	`8000`

Il CORS dell'API è configurato esplicitamente per accettare richieste da `http://localhost:8501` e `http://127.0.0.1:8501` (il frontend Streamlit di default).

Istruzioni di avvio complete: vedi AVVIO.md.

---

7. Funzionalità implementate

Autenticazione

- Registrazione e login studente (password con hash SHA-256).
- Login persistente tramite cookie di sessione (token opaco salvato lato browser + DB).
- Logout esplicito con invalidazione del token lato server.

Gestione esami e materiali

- Creazione, rinomina, eliminazione esami.
- Upload di uno o più PDF per esame, con estrazione testo e indicizzazione RAG automatica.
- Visualizzazione dei materiali caricati per esame (pagine, numero di chunk).
- Eliminazione esame → cascata su sessioni/materiali nel DB + rimozione della collection ChromaDB.

Generazione quiz

- Generazione di N domande aperte (3-10) dal materiale indicizzato, su tre livelli di difficoltà (base / intermedia / avanzata), con retry automatico finché il modello non produce un JSON valido e completo.

Tutor socratico (sessione quiz)

- Domanda → prima risposta → domanda guida socratica (mai la risposta corretta) → seconda risposta → valutazione 0-10 con motivazione, punti corretti/mancanti e valutazione del miglioramento.
- Guardrail automatici: punteggio azzerato per non-risposte, punteggio corretto per coerenza quando non sono segnalate lacune reali.
- Contestazione del voto con rivalutazione da parte di un agente imparziale dedicato.

Chatbot dubbi liberi

- Lo studente scrive un dubbio proprio (non legato al quiz) su un argomento del materiale caricato.
- Il tutor risponde con una domanda socratica di apertura (mai la risposta diretta).
- Dopo la risposta dello studente, mostra punti corretti/mancanti e la **risposta corretta e completa** al dubbio originale.
- Interfaccia a chat reale (bolle di conversazione) nella pagina `app/pages/chat.py`.
- Gestione esplicita del caso “nessun materiale pertinente indicizzato”.

Motore adattivo

- Tracciamento del punteggio medio per argomento durante la sessione.
- Scelta della prossima domanda con priorità: argomenti deboli → argomenti mai visti → qualunque domanda rimanente.
- Calcolo di argomenti deboli/forti e generazione di un report di sessione aggregato.

Report e appunti

- Report di fine sessione in Markdown e PDF (media, argomenti da rivedere/padroneggiati, verdetto).
- Rigenerazione del PDF a posteriori dallo storico se il file originale manca.
- Generazione di appunti sintetici in Markdown/PDF a partire dal materiale caricato (per esame o per singolo file) oppure dal log di una sessione già svolta (con focus sulle lacune rilevate).

Storico

- Elenco sessioni per esame con statistiche aggregate (media, miglior voto, sessioni completate).
- Dettaglio Q&A per sessione (prima/seconda risposta, domanda guida, motivazione, esito eventuale contestazione).
- Gestione sessioni “orfane” (avviate ma mai concluse): chiusura manuale o eliminazione.
- Download del report PDF e generazione appunti sintetici direttamente dallo storico.

API REST (parallela alla UI)

- Endpoint completi per autenticazione, gestione esami/ingest PDF, ciclo di vita sessione (creazione, generazione quiz, ogni step del tutor socratico, prossima domanda adattiva, chiusura e report, dettaglio/lista sessioni).

- Documentazione automatica OpenAPI su `/docs` e `/redoc`.
- Health check su `/health`.

Qualità e test

- 157 test automatici: unit (logica pura di ogni agente), integration (router FastAPI con DB e ChromaDB isolati), e2e (interazione reale con le pagine Streamlit via `AppTest`, click sui bottoni compresi).
- Nessuna chiamata LLM reale nei test (`FakeAgent` dedicato) e nessun dato di test scritto sul DB/ChromaDB/reports reali del progetto.

---

8. Struttura delle cartelle

```
studycoach/
├── app/                          # Frontend Streamlit
│   ├── main.py                   # Entry point, routing tra pagine, CSS globale
│   └── pages/
│       ├── login.py               # Login / registrazione
│       ├── upload.py              # Lista esami, creazione esame
│       ├── exam.py             # Dettaglio esame: avvia sessione, upload PDF, appunti
│       ├── session.py             # Loop quiz + tutor socratico
│       ├── chat.py                # Chatbot dubbi liberi (DoubtTutor)
│       └── history.py             # Storico sessioni, report, appunti
│
├── backend/
│   ├── agents/                   # Logica di dominio (indipendente dall'interfaccia)
│   │   ├── ingestor.py             # Estrazione PDF, chunking, embedding, RAG
│   │   ├── quiz_generator.py       # Generazione domande dal materiale
│   │   ├── socratic_tutor.py       # SocraticTutor + DoubtTutor (loop socratico)
│   │   ├── adaptive_engine.py      # Selezione adattiva delle domande, statistiche
│   │   ├── notes_generator.py      # Appunti sintetici (Markdown + PDF)
│   │   ├── report_generator.py     # Report di sessione (Markdown + PDF)
│   │   └── prompts/                # Template dei prompt LLM, versionati
│   ├── api/                        # API REST FastAPI
│   │   ├── main.py                 # App FastAPI, CORS, router, health check
│   │   ├── auth.py                 # Endpoint autenticazione
│   │   ├── exams.py                # Endpoint esami/ingest
│   │   └── sessions.py             # Endpoint ciclo di vita sessione/tutor
│   ├── db/
│   │   └── models.py               # Modelli SQLAlchemy + inizializzazione DB
│   ├── models/
│   │   └── schemas.py              # Schemi Pydantic per l'API
│   └── session_cookie.py           # Login persistente via cookie browser
│
├── test/                           # Suite di test (unit + integration + e2e)
├── data/                 # DB SQLite, ChromaDB, PDF caricati (runtime, non versionato)
└── reports/                        # Report PDF/Markdown generati (runtime)
```

---

9. Testing

La suite copre l'intero stack su tre livelli, tutti eseguibili con un unico comando (`pytest`), completamente isolati da dati reali:

Livello	Cosa copre	Come
Unit	Logica pura di ogni agente: guardrail di valutazione, parsing/riparazione JSON, motore adattivo, helper di pulizia testo/PDF, validatori Pydantic, token di sessione, modelli DB (cascade delete, vincoli)	Nessun I/O esterno dove possibile
Integration	Router FastAPI reali (`auth`, `exams`, `sessions`) tramite `TestClient`, incluso upload/ingest di un PDF vero e il flusso completo quiz → tutor → report	DB SQLite temporaneo, collection ChromaDB temporanea con embedding finti deterministici
E2E	Interazione reale con le pagine Streamlit (`app/main.py` e `app/pages/*`) — click sui bottoni, compilazione form, chat — tramite `streamlit.testing.v1.AppTest`	Stesso isolamento DB/ChromaDB, più `FakeAgent` al posto delle chiamate LLM reali

Fixture condivise in `test/conftest.py` garantiscono che **nessun test scriva mai** sul database, sulla collection ChromaDB o sulla cartella `reports/` reali del progetto.

---

10. Limitazioni note e possibili sviluppi futuri

- Lo stato dei tutor/engine attivi nell'API REST è **in memoria di processo**: un riavvio del server interrompe le sessioni socratiche in corso (il punteggio già assegnato resta comunque salvato su DB).
- L'estrazione testo dai PDF (`pypdf`) non gestisce PDF scansionati/immagine senza testo selezionabile (nessun OCR).
- L'autenticazione dell'API REST non emette JWT: è pensata per essere consumata dal frontend Streamlit tramite `session_state`; un client esterno andrebbe protetto con un middleware di autenticazione dedicato.
- Sviluppi possibili: persistenza dello stato tutor/engine (es. Redis o tabella dedicata) per sopravvivere ai riavvii; OCR per PDF scansionati; autenticazione JWT per l'API REST pubblica.

11. Divisione dei contributi (Agent)

Come richiesto, di seguito viene indicato esplicitamente quale sviluppatore ha realizzato ciascun agente della cartella `backend/agents/`, in aggiunta alla suddivisione generale già descritta nella sezione 7 (Funzionalità implementate).

Agent (file)	Ruolo nell'agente	Sviluppatore
`ingestor.py`	Estrazione testo dai PDF, chunking (500 parole, overlap 50) ed embedding locale (sentence-transformers); indicizzazione RAG in ChromaDB.	Diabi Mohamed Zakaria
`quiz_generator.py`	Recupero dei chunk più rilevanti (RAG) e generazione di N domande aperte calibrate su tre livelli di difficoltà.	Gouaiche Kamal
`socratic_tutor.py`	Classi SocraticTutor e DoubtTutor: loop domanda → risposta → domanda guida → valutazione, guardrail di coerenza/severità, contestazione del voto.	Diabi Mohamed Zakaria
`adaptive_engine.py`	Tracciamento dei punteggi per argomento e selezione della prossima domanda, con priorità agli argomenti deboli.	Gouaiche Kamal
`notes_generator.py`	Generazione di appunti sintetici (Markdown + PDF) dal materiale caricato o dal log di una sessione già svolta.	Diabi Mohamed Zakaria
`report_generator.py`	Generazione del report di fine sessione (Markdown + PDF): media, argomenti deboli/forti e verdetto.	Gouaiche Kamal

Riepilogo per sviluppatore

- Diabi Mohamed Zakaria — ingestor.py, socratic_tutor.py, notes_generator.py (3 agent)
- Gouaiche Kamal — quiz_generator.py, adaptive_engine.py, report_generator.py (3 agent)

Gli altri livelli del progetto (frontend Streamlit, API REST, persistenza dati, testing) restano attività condivise tra i due sviluppatori, come già descritto nelle sezioni precedenti.

12. Utilizzo di intelligenza artificiale

Le fasi di lavoro in cui è stata utilizzata l'intelligenza artificiale sono le seguenti:

- GitHub Copilot (piano Student) — usato con selezione automatica del modello (autoselect), principalmente per la finalizzazione della parte grafica dell'interfaccia e per l'individuazione di bug ed errori nel codice durante lo sviluppo.
- Claude (chat) — usata nella fase iniziale del progetto come supporto per costruire un'idea di massima su come impostare l'applicazione, prima ancora di iniziare a scrivere codice: ci ha aiutato a ragionare sull'architettura generale (suddivisione in frontend, API REST e livello di agenti) e a capire come usare il framework Agno e la libreria Streamlit, due tecnologie non utilizzate in precedenza da noi.